

Numerikus Számítások

1. előadás

Csókás Eszter

Szegedi Tudományegyetem

Informatika Intézet

Számítógépes Optimalizálási Tanszék



Adminisztráció

- email: `csokas@inf.u-szeged.hu`
 - anyagok: [honlapon](#)
 - kérdések: coospace fórum
 - fogadóóra: hétfő 17:00-18:00, Árpád tér 2. 49-es iroda (email-es egyeztetés szükséges)
 - társelőadó: Dr. Homolya Viktor (`homolyav@inf.u-szeged.hu`)
-
- Előadás: szóbeli vagy írásbeli vizsga, beugróval
 - Gyakorlat: folyamatos számonkérés, további részletek a gyakorlaton

Követelmények, teljesítés feltételei

Vizsga anyaga:

- előadáshoz tartozó diasor
- az előadáson elhangzott anyag
- egyszerű ismétlő kérdések a gyakorlati anyagból

Vizsga menete:

- a gyakorlatot sikeresen teljesítő hallgatók vizsgázhatnak
- a vizsga első része a beugró, ami egy coospace teszt - elérendő minimum 60%
- sikeres beugró után a vizsga második része írásbeli vagy szóbeli vizsga

Követelmények, teljesítés feltételei

Írásbeli vagy szóbeli vizsga:

- a hallgató a sikeres beugró után választhat, hogy írásban vagy szóban szeretne vizsgázni
- az írásbeli vizsgán csak **elégséges (2)** vagy **közepes(3)** osztályzat szerezhető
 - az írásbeli vizsgán kifejtős kérdések lesznek, melyre összefüggő, egész mondatokból álló válaszokat várunk
 - érdemes a kiadott tételsor alapján készülni, mert az lefedi a teljes anyagot
- mindenképp szóbelizni kell annak, aki **jó (4)** vagy **jeles (5)** osztályzatot szeretne, de szóbeli vizsgán **bármely érdemjegy** megszerezhető
 - szóbeli vizsgán sokszor előfordul, hogy segítő kérdések által a vizsgázónak több minden jut eszébe, mint az üres lapot nézve
 - tételsor alapján, felkészülési idő után kell 2 tételt röviden kifejteni
 - bármilyen jegy megszerezhető: így ha "rossz tételt" is kapsz, kérdezgetések útján segítjük a vizsgát

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

De nem úgy, hogy adott valamilyen képlet és abba egyszerűen számokat helyettesítünk.

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

De nem úgy, hogy adott valamilyen képlet és abba egyszerűen számokat helyettesítünk.

Legtöbbször ilyenkor inkább valamilyen **algoritmus** végrehajtásáról van szó.

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

De nem úgy, hogy adott valamilyen képlet és abba egyszerűen számokat helyettesítünk.

Legtöbbször ilyenkor inkább valamilyen **algoritmus** végrehajtásáról van szó.

Matematikai problémák számos esetben NEM oldhatók meg elemi műveletek véges sorozatával.

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

De nem úgy, hogy adott valamilyen képlet és abba egyszerűen számokat helyettesítünk.

Legtöbbször ilyenkor inkább valamilyen **algoritmus** végrehajtásáról van szó.

Matematikai problémák számos esetben NEM oldhatók meg elemi műveletek véges sorozatával.

Ami inkább történik: **gyors** algoritmusok a megoldás egy **közelítő** értékéhez konvergálnak.

Kezdjük az elején: mi ez?

Számolási feladatokat leggyakrabban számítógéppel oldunk meg.

De nem úgy, hogy adott valamilyen képlet és abba egyszerűen számokat helyettesítünk.

Legtöbbször ilyenkor inkább valamilyen **algoritmus** végrehajtásáról van szó.

Matematikai problémák számos esetben NEM oldhatók meg elemi műveletek véges sorozatával.

Ami inkább történik: **gyors** algoritmusok a megoldás egy **közelítő** értékéhez konvergálnak.

A numerikus számítások során a folytonos matematikához kapcsolódó problémák megoldására szolgáló algoritmusok vizsgálata.

Miért van szükségünk erre?

Nézzük meg a következő, nagyon egyszerű példát:

$$x1 = 0.3$$

$$x2 = 0.1 + 0.1 + 0.1$$

$$x1 == x2$$

Miért van szükségünk erre?

Nézzük meg a következő, nagyon egyszerű példát:

$$x1 = 0.3$$

$$x2 = 0.1 + 0.1 + 0.1$$

$$x1 == x2$$

Az `x1 == x2` megnézi, hogy egyenlőek-e és egy logikai `true/false` a válasz.

Itt nem azt kapjuk amire számítunk, ugyanis a MATLAB a kirítékelés során `false` eredményt ad vissza.

Miért lehet ez? Visszatérünk rá a félév során!

Menjünk ki a való világba: Ariane 5 hordozórakéta

7 milliárd dolláros költségvetés

1996. június 4, 37 másodperccel a kilövés után:

https://www.youtube.com/watch?v=N6PWATvLQCY&ab_channel=SciNews

Tehetetlenségi navigációs rendszer: külső segítség nélkül képes hordozóeszköze helyzetének, haladási irányának és sebességének megállapítására

A rakéta önmegsemmisítést hajtott végre indítás után 37 mp-el az irányító szoftver hibája miatt:
64 bites lebegőpontos szám $\Rightarrow$ 16 bites egész, így túlcsordult

Ez volt a történelem egyik legdrágább "numerikus hibája".

The Google Lambda logo is displayed in the center of the slide. It features the word "Google" in its signature multi-colored font (blue 'G', red 'o', yellow 'o', blue 'g', green 'l', red 'e'), followed by the word "Lambda" in a blue, monospace-style font.

Számrendszerek

A tízes számrendszer

Alapszáma: **10**

Mit jelent ez?

- számjegyek: 0-tól 9-ig (10 db)
- helyiértékek: 10^i , $i = 0, \dots, n$

Egy számrendszer legnagyobb számjegye mindig az (alapszám - 1).



A tízes számrendszer

Alappélda: $4237,8_{10} =$

helyiértékek:	3.	2.	1.	0.	-1.
sámjegyek:	4	2	3	7	8

$$\begin{aligned} 4237,8_{10} &= 4 \cdot 10^3 + 2 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0 + 8 \cdot 10^{-1} = \\ &= 4000 + 200 + 30 + 7 + 0.8 \end{aligned}$$

Ne felejtsük: $10^0 = 1$, általánosan: $x^0 = 1$

Miért használjunk a megszokottól (10-es) eltérő számrendszert?

Mert más csoportosítási egység praktikusabb...

- mondjuk kettes csoportosításokat használni

Szakmai példa:

- bitek ábrázolása: digitális kommunikáció, memória, regiszterek, stb.
 - 2-es számrendszerben praktikusabb: **1/0**, a bitek ezeket az értékeket vehetik fel
- byte-ok ábrázolása (8 bites egység): memória, regiszterek, stb.
 - 16-os számrendszerben praktikusabb (lásd később)

Általános felírása egy számnak

Legyen A egy szám

- ennek egész számjegyei a_i , tizedes számjegyei b_i
- i a helyiérték
- q alapú a számrendszer

A szám ekkor így néz ki:

$$a_n a_{n-1} \dots a_0, b_1 b_2 \dots b_m$$

$$\begin{aligned} & a_n a_{n-1} \dots a_0, b_1 b_2 \dots b_m = \\ & = a_n \cdot q^n + a_{n-1} \cdot q^{n-1} \dots a_0 \cdot q^0 + b_1 \cdot q^{-1} + b_2 \cdot q^{-2} + \dots + b_m \cdot q^{-m} \end{aligned}$$

A kettes számrendszer

Alapszáma: 2

Mit jelent ez?

- számjegyek: 0, 1 (2 db)
- helyiértékek: 2^i , $i = 0, \dots, n$

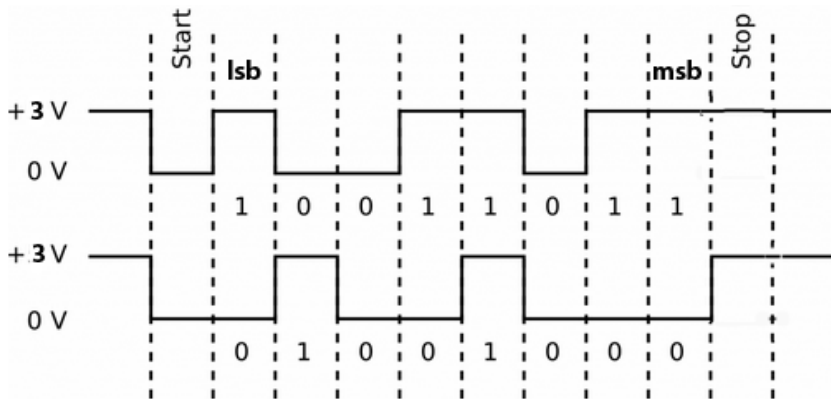
A kettes számrendszer

Alappélda: $10000101_2 =$

helyiértékek:	7.	6.	5.	4.	3.	2.	1.	0.
sámjegyek:	1	0	0	0	0	1	0	1

$$\begin{aligned} 10000101_2 &= 1 \cdot 2^7 + 1 \cdot 2^2 + 1 \cdot 2^0 = \\ &= 2^7 + 2^2 + 2^0 = 128 + 4 + 1 = 133_{10} \end{aligned}$$

Példa: soros kommunikáció, digitális adat átvitel



A tizenhatos számrendszer

Alapszáma: 16

Mit jelent ez?

- számjegyek: 0-tól 15-ig *DE ez meg hogy lehet?*
 - Például a 15 az nem is számjegy! Ezért jönnek a betűk:
 - számjegyek: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F (16 db)
- helyiértékek: 16^i , $i = 0, \dots, n$

A tizenhatos számrendszer

Alappélda: $85_{16} =$

helyiértékek:	1.	0.
sámjegyek:	8	5

$$\begin{aligned} 85_{16} &= 8 \cdot 16^1 + 5 \cdot 16^0 = \\ &= 8 \cdot 16 + 5 \cdot 1 = \underline{133}_{10} \end{aligned}$$

Összefüggések a 2-es és 16-os számrendszer ábrázolása között

Korábbi példánk

$$1000\ 0101_2 = 133_{10}$$

$$85_{16} = 133_{10}$$

$$10000101_2 = 1 \cdot 2^7 + 1 \cdot 2^2 + 1 \cdot 2^0 = 2^7 + 2^2 + 2^0 = 128 + 4 + 1 = 133_{10}$$

$$85_{16} = 8 \cdot 16^1 + 5 \cdot 16^0 = 8 \cdot 16 + 5 \cdot 1 = 133_{10}$$

$$1000\ 0101_2$$

$$\rightarrow 1000_2 = 8_{10} = 8_{16}$$

$$\rightarrow 1 \cdot 2^3$$

$$\rightarrow 0101_2 = 5_{10} = 5_{16}$$

$$\rightarrow 1 \cdot 2^0 + 1 \cdot 2^2$$

$$1000\ 0101_2 = 85_{16}$$

$$8 \cdot 5$$

További példa: RGB színekódolás

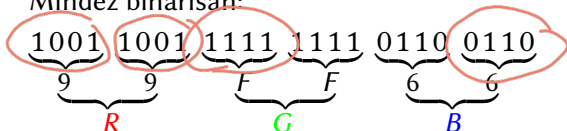
24 biten, 3 byte-on

Most nézzük a RGB(153, 255, 102)-t!

Átalános felírás: $0x\ RR\ GG\ BB$

- $153_{10} = 99_{16} = 0x99$
 - $255_{10} = FF_{16} = 0xFF$
 - $102_{10} = 66_{16} = 0x66$
- } $0x99FF66$

Mindez binárisan:



Műveletek

Átszámítási módok

- 2-es számrendszerből 10-es számrendszerbe ✓
- 16-os számrendszerből 10-es számrendszerbe ✓

Nézzük 10-es számrendszerből 2-es számrendszerbe az átváltást:

Feladat: $131_{10} = ?_2$

10000011_2

131_{10}	
65	1
32	0
16	0
8	0
4	0
2	0
1	1
0	

Összeadás (idén nem vesszük részletesen)*

Ahogy azt általános iskolában megtanultuk 10-es számrendszerbe, úgy működik a többiben is...

Példa (10-es számrendszerben): $1466_{10} + 188_{10}$:

$$\begin{array}{r} 1 \ 4 \ 6 \ 6 \\ + \quad 1 \ 8 \ 8 \\ \hline 1 \ 6 \ 5 \ 4 \end{array}$$

Példa (16-os számrendszerben): $5BA_{16} + BC_{16}$:

$$\begin{array}{r} 5 \ B \ A \\ + \quad B \ C \\ \hline 6 \ 7 \ 6 \end{array}$$

Szorzás (idén nem vesszük részletesen)*

Hasonlóan az összeadáshoz, itt is kiterjeszthető a 10-es számrendszerben tanultak. Kérdés, hogy megéri-e más számrendszerben számolni vagy váltsuk át és úgy végezzük el a művelet, aztán váltsuk vissza? Próbáljuk ki!

Példa (16-os számrendszerben): $56_{16} \cdot C_{16}$:

	5	6	·	C
	0	4	8	$(6 \cdot C)$
+	3	C		$(5 \cdot C)$
	4	0	8	

Számábrázolási formátumok

Motiváció:

- értsük meg, hogy mit csinál a számítógép
 - alapvetés: a számítógép bináris számábrázolást használ
- (hardverközel) programozás során gyakran térünk át a különböző számábrázolási formátumok között (kasztolás)
- ez akkor működik jól, ha tudjuk, hogy a kezelt adataegységekben, mely bitcsoportok, milyen célt szolgálnak
- számábrázolási formátumok pontossága és korlátai elengedhetetlenek a hibák elkerüléséhez

A számok kétféle nagy csoportját különítjük el:

- az egyszerűbb, egész típusú (integer) számok
- a nem egész (tört) számok
- a számok nagysága vagy a pontossága miatt a valós (real) számok

Tények:

- egész számokkal való műveletvégzés általában gyorsabban elvégezhető
- lebegőpontos számokkal történő műveletek számítása időigényes és más bajok is vannak
- ezért sokszor használjuk őket még törtszámok kifejezésére
- *de ez hogy lehet?*

Számábrázolás

- fixpontos számábrázolás:

- úgy tudunk törtszámokat kifejezni, hogy a számba egy bináris (kettedes) pontot "képzünk"
- a pont mozgását korrigáljuk eltolással, amikor műveleteket végzünk el
- bizonyos mértékben segít, de az alapvető problémán nem:
a számok nagysága tág határok között mozognak

3_1 → $3 \cdot 10^0$ → $5 \cdot 10^7$
 $5.000.000$

- lebegőpontos számábrázolás:

- az ábrázolás pontosságát a szám tényleges nagyságához kellene igazítani
- azaz egy pontosság (pl: "kéttizedes pontosságú szám") nem mond semmit a szám nagyságának ismerete nélkül
- ezt a problémát a matematika a normál alak használatával hidalja át
- az informatika, vagy sokkal inkább számítástechnika világában pedig a lebegőpontos számábrázolást használjuk

A tört számok világa

Gondoljuk át a következő problémákat:

- számítógépen egy törtszámot véges számú biten tudunk ábrázolni és ennek az ábrázolásnak van egy véges pontossága, számábrázolási nagysága; míg a törtszám, mint matematikai értelemben vett valós szám, ezt nem tartalmazza
- véges törtrészű tizedes törtek a kettes számrendszerben történő ábrázolás miatt nem biztos, hogy véges bitszámon lesz ábrázolható
 - *a 0.1 véges tört a tízes számrendszerben, de mi a helyzet a kettes számrendszerben?*

A tört számok világa

Gondoljuk át a következő problémákat:

- számítógépen egy törtszámot véges számú biten tudunk ábrázolni és ennek az ábrázolásnak van egy véges pontossága, számábrázolási nagysága; míg a törtszám, mint matematikai értelemben vett valós szám, ezt nem tartalmazza
- véges törtrészű tizedes törtek a kettes számrendszerben történő ábrázolás miatt nem biztos, hogy véges bitszámon lesz ábrázolható
→ *a 0.1 véges tört a tízes számrendszerben, de mi a helyzet a kettes számrendszerben?*

Emlékszünk erre az egyszerű példára? Van már sejtésünk, hogy miért hamis a kifejezés?

$$x1 = 0.3$$

$$x2 = 0.1 + 0.1 + 0.1$$

$$x1 == x2$$

A tört számok világa

Gondoljuk át a következő problémákat:

- számítógépen egy törtszámot véges számú biten tudunk ábrázolni és ennek az ábrázolásnak van egy véges pontossága, számábrázolási nagysága; míg a törtszám, mint matematikai értelemben vett valós szám, ezt nem tartalmazza
- véges törtrészű tizedes törtek a kettes számrendszerben történő ábrázolás miatt nem biztos, hogy véges bitszámon lesz ábrázolható
 - *a 0.1 véges tört a tízes számrendszerben, de mi a helyzet a kettes számrendszerben?*
- a számítógépek műveletvégzése gyors, nagyságrendekkel több darab műveletet tudunk velük elvégezni, mint "emberi" műveletvégzés során, így egymásra épülő műveletek akár milliárdjait is el tudjuk végezni
 - *mekkora lehet a számítás végére a hiba, ha egyik műveletünk sem pontos, még elvileg sem?*

IEEE Std. 754-2008

Ezek miatt a lebegőpontos számok kezelését egységesítették:

- az amerikai az Institut of Electrical and Electronics Engineer (IEEE) szabványosította IEEE Std. 754-1985 néven, 1985-ben
- ennek kiterjesztése a IEEE Std. 754-2008 néven jelent meg
 - megtartotta az eredeti formátumokat
 - hozzáigazította az ábrázolási méreteket a mai buszméretekhez (128 bit a korábbi 80/96 bit helyett)